EXP 24

```
clc;
clear;
close all;

%% 1. System Parameters
Nbits = 100000;          % Number of transmitted bits
SNR_dB = 0:2:20;         % SNR range
fc = 100e3;              % Carrier frequency (Hz)
Fs = 1e6;                % Sampling frequency (Hz)
Rs = 50e3;               % Symbol rate (symbols/sec)
Ts = 1/Rs;

% QPSK requires two bits per symbol
Nbits = 2*floor(Nbits/2);
rng(1);

%% 2. Generate Random Binary Data
bits = randi([0 1], Nbits, 1);

% Serial-to-parallel conversion
bitPairs = reshape(bits, 2, []).';

% QPSK mapping with unit average symbol power
txSymbols = ((1 - 2*bitPairs(:,1)) + ...
    1j*(1 - 2*bitPairs(:,2))) / sqrt(2);

Nsym = length(txSymbols);

%% 3. Passband Modulation
```

```
% Each symbol is represented by one sample in this
% simplified passband-equivalent simulation.
t = (0:Nsym-1)' * Ts;

txPassband = real(txSymbols .* exp(1j*2*pi*fc*t));

figure;
plot(t(1:200)*1e3, txPassband(1:200));
xlabel('Time (ms)');
ylabel('Amplitude');
title('Transmitted QPSK Passband Signal');
grid on;

%% 4. Multipath Rayleigh Fading Channel
% Three-tap discrete-time channel with delayed paths
h = [0.9, 0.45, 0.2] .* ...
    exp(1j*[0, 0.7, -1.1]);

h = h / norm(h);

% Complex baseband equivalent multipath channel
rxChannel = filter(h, 1, txSymbols);

%% 5. BER Without Equalization
BER_noEQ = zeros(size(SNR_dB));
BER_EQ = zeros(size(SNR_dB));

% Zero-forcing equalizer for the three-tap channel
% For each symbol, use a linear MMSE equalizer
% with a fixed channel model.
L = 11;
```

```
H = convmtx(h(:), L);
H = H(1:L+length(h)-1, :);

% Select the center symbol response as the target
delay = 2;
d = zeros(size(H,1), 1);
d(delay+1) = 1;

%% 6. Add AWGN, Equalize and Calculate BER
for k = 1:length(SNR_dB)

   snrLinear = 10^(SNR_dB(k)/10);

   % Noise variance based on average received signal power
   noisePower = mean(abs(rxChannel).^2) / snrLinear;

   noise = sqrt(noisePower/2) * ...
      (randn(size(rxChannel)) + 1j*randn(size(rxChannel)));

   rx = rxChannel + noise;

   %% A. Detection Without Equalization
   % Simple one-tap channel compensation using main path
   rxNoEQ = rx / h(1);

   detectedNoEQ = zeros(Nsym, 2);
   detectedNoEQ(:,1) = real(rxNoEQ) < 0;
   detectedNoEQ(:,2) = imag(rxNoEQ) < 0;

   bitsNoEQ = reshape(detectedNoEQ.', [], 1);
   BER_noEQ(k) = mean(bitsNoEQ ~= bits);
```

```
%% B. MMSE Equalization
% Construct a linear MMSE equalizer
% Using the channel impulse response
p = 2;
Hf = zeros(L, L);

for row = 1:L
   for col = 1:L
      idx = row - col + 1;
      if idx >= 1 && idx <= length(h)
         Hf(row,col) = h(idx);
      end
   end
end

% Regularized inverse for noise robustness
w = (Hf' * Hf + noisePower * eye(L)) \ ...
   (Hf' * eye(L));

% Equalize received signal using a simple linear filter
rxEQ = filter(conj(flipud(w(:,p))), 1, rx);

% Compensate for equalizer delay approximately
rxEQ = rxEQ(3:end);
txRef = txSymbols(1:length(rxEQ));

detectedEQ = zeros(length(rxEQ), 2);
detectedEQ(:,1) = real(rxEQ) < 0;
detectedEQ(:,2) = imag(rxEQ) < 0;
```

```
    bitsEQ = reshape(detectedEQ.', [], 1);
    bitsRef = reshape([real(txRef)<0, imag(txRef)<0].', [], 1);

    nCompare = min(length(bitsEQ), length(bitsRef));
    BER_EQ(k) = mean(bitsEQ(1:nCompare) ~= bitsRef(1:nCompare));
end

%% 7. Plot BER Performance
figure;
semilogy(SNR_dB, max(BER_noEQ, 1/Nbits), 'ro-', ...
    'LineWidth', 1.5);
hold on;
semilogy(SNR_dB, max(BER_EQ, 1/Nbits), 'bs-', ...
    'LineWidth', 1.5);
grid on;

xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('QPSK BER under Multipath Fading');
legend('Without Equalization', 'With Equalization', ...
    'Location', 'southwest');

%% 8. Display Results
fprintf('\nQPSK Multipath Fading Simulation Results\n');
fprintf('----------------------------------------\n');

for k = 1:length(SNR_dB)
    fprintf('SNR = %2d dB | BER without EQ = %.6f | BER with EQ = %.6f\n', ...
        SNR_dB(k), BER_noEQ(k), BER_EQ(k));
end
```

fprintf('\nSimulation completed.\n');

